Does Momentum Change What CNNs Learn in the First Layer?

Markus Massina
Binghamton University
GitHub: Markus112105

Michael Truong
Binghamton University
GitHub: Michael-Truong111

***Abstract*—Chowers and Weiss showed that the first layer of a convolutional neural network learns a consistent energy profile across random seeds and architectures, and with random labels. They explained it with a formula derived for plain gradient descent, and their theory does not cover momentum. We ask whether adding momentum to SGD changes the first-layer energy profile, the test accuracy, or the training time of VGG11-BN on CIFAR-10. We trained six models for 30 epochs, three seeds with plain SGD and the same three seeds with momentum 0.9. Both runs of a seed share the same initial weights and batch order. Momentum raised mean test accuracy from 54.54% to 78.56% and took 1% longer to train. The energy profiles stayed consistent across seeds, with a mean correlation of 0.92 for plain SGD and 0.97 with momentum. On the same seed, the profiles of the two optimizers had a correlation of 0.95 to 0.99. Momentum changed the training speed and the test accuracy, and the first-layer energy profile kept a correlation of at least 0.95 with the plain SGD profile.**

***Index Terms*—convolutional neural networks, first layer, energy profile, momentum, stochastic gradient descent, CIFAR-10**

I. Introduction

The first convolutional layer of a CNN applies its filters to the input pixels. In VGG11 each filter covers a 3×3 patch of three color channels. Chowers and Weiss [1] measure this layer with an energy profile and report that the profile is consistent across training runs. On CIFAR-10, VGG11 models trained from different seeds have a profile correlation of 0.99, and a trained model has a correlation of -0.13 with its own initialization. They also derive a formula for the profile of a linear CNN trained with gradient descent. The formula depends on the statistics of image patches, the step size, and the number of steps, and the labels do not appear in it.

The derivation assumes plain gradient descent. The authors state that momentum, weight decay, and learning-rate schedules are outside their theory, and their comparison with pretrained models changes all three together [1]. We isolate momentum, a standard addition to SGD for deep networks [2]. Our question is whether adding momentum 0.9 to SGD changes the first-layer energy profile, the test accuracy, or the training time of VGG11 with batch normalization (VGG11-BN) [3], [4] on CIFAR-10 [5].

We train six models for 30 epochs, three seeds for each optimizer, with the same initial weights and batch order within a seed. Momentum raises mean test accuracy from 54.54% to 78.56% and adds 1% to the training time. On the same seed, the energy profiles of the two optimizers have a correlation of 0.95 to 0.99.

II. Background

A. Energy Profile

Let $u_1, \dots, u_{27}$ be the principal components of $3 \times 3 \times 3$ patches taken from training images, and let $w_1, \dots, w_K$ be the first-layer filters, each flattened to 27 values. The energy along component i is

$$e_i = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \left(w_k^\top u_i \right)^2}. \tag{1}$$

The energy profile is the vector $(e_1, \dots, e_{27})$ divided by its largest entry. Reordering or rescaling the filters leaves the profile unchanged [1], so it can compare two networks that store the same filters in a different order. The similarity of two networks is the Pearson correlation between their profiles.

B. Reported Results

For VGG11 on CIFAR-10, [1] reports a correlation of 0.99 between seeds and -0.13 ± 0.18 between a trained model and its initialization. The correlation is 0.87 between VGG11 and ResNet. It is 0.90 ± 0.02 between a model trained on the true labels and one trained on random labels. Random labels carry no class information. The authors conclude that the patch statistics and the optimizer set the profile, and that the labels have little effect.

C. Theory

The theory in [1] covers a two-layer linear CNN trained with gradient descent on the squared loss, with step size η for t steps. The derived profile is

$$e_i \propto \lambda_i \, \frac{|1 - (1 - a_i)^t|}{a_i}, \qquad a_i \propto \eta \lambda_i^2, \tag{2}$$

where λ_i^2 is the mean squared projection of the training patches on u_i. With t fitted, this formula has a correlation of 0.94 ± 0.01 with the profile of VGG11 trained on CIFAR-10. As t grows, the formula approaches a whitening transform, which removes the correlation between neighboring pixels [1].

Equation (2) has no momentum term. The fitted formula has a correlation of 0.91 with a pretrained CIFAR-10 VGG11 and 0.69 with a pretrained ResNet20. Their training used

momentum, weight decay, and a learning-rate schedule, and the authors point out that their theory covers none of the three [1].

III. Methodology

We keep the measurement of [1] and change one argument of the optimizer.

A. Measurement

For each run we store the first-layer weights before training, W_0, and after training, W_T. We apply (1) to the difference $W_T - W_0$, as the authors' code does, so the profile measures the change that training produced. VGG11-BN has $K = 64$ first-layer filters. All six runs share one PCA basis.

We compute nine correlations. Three are between seeds for plain SGD, three are between seeds for momentum, and three are between the two optimizers on the same seed. The first group repeats the seed comparison of [1]. The third group isolates the effect of momentum, because the two runs of a seed differ only in μ.

B. Optimizer

Plain SGD updates the weights w with the minibatch gradient g as $w \leftarrow w - \eta g$. SGD with momentum μ keeps a velocity v and updates

$$v \leftarrow \mu v + g, \qquad w \leftarrow w - \eta v. \tag{3}$$

Setting $\mu = 0$ gives plain SGD. If g stays constant, v converges to $g/(1-\mu)$, so $\mu = 0.9$ takes a step up to 10 times larger at the same η.

C. Hypotheses

- H1: With momentum, the energy profile stays consistent across seeds.
- H2: On the same seed, the two optimizers learn similar profiles.
- H3: Momentum reaches a higher test accuracy in 30 epochs.
- H4: Momentum does not noticeably increase the training time.

IV. Our Contribution

The original repository has one utility file and two notebooks. Its training function, `train_model`, uses plain SGD, requires a CUDA device, and returns no measurements.

A. Compatibility

We rewrote the utility file so the code selects a CUDA GPU, an Apple GPU, or the CPU, and no longer needs the torchmetrics package. `train_model` now returns the loss and accuracy of every epoch on the training set and the test set. We moved the patch PCA from the notebooks into one function that accumulates a 27×27 matrix per batch, so it never stores the 9,000,000 patches. We pinned the model definition [7] to one commit.

B. Experiment Options

We added three options to `train_model`.

1) A `momentum` argument that we pass to the PyTorch SGD optimizer [6]. The default is 0, which keeps plain SGD.
2) A timer around the training part of each epoch. The timer waits for queued GPU work to finish before it starts and before it stops.
3) An option to evaluate the test set only after the last epoch. The test set then plays no part in training, and evaluation time stays out of the measured time.

Comments in the source mark these additions.

C. Tests and Notebook

We wrote nine unit tests. Four cover the three new options. Five cover the training metrics, the test loader, the PCA, the energy profile, and the formula fit.

Our new notebook, `optimizer_experiment.ipynb`, runs the six configurations and saves one file per run with the settings, software versions, per-epoch history, initial and final first-layer weights, and energy profile. It then checks the saved runs, computes the statistics in Table I, and draws Fig. 1.

M. Massina wrote the tests, the local setup, and notebook sections 1, 2, and 5, and ran the experiment on Colab. M. Truong wrote notebook sections 3, 4, 6, and 7 and analyzed the results. Both authors planned the project.

V. Experimental Setup

A. Data and Model

CIFAR-10 [5] has 50,000 training images and 10,000 test images of 32×32 pixels in 10 classes. We normalize each color channel and use no augmentation. The model is VGG11-BN from [7] at a pinned commit, with random initial weights.

B. Training

Every run uses the cross-entropy loss, learning rate 0.01, batch size 256, and 30 epochs. We use no weight decay, no learning-rate schedule, and no Nesterov update. The only setting that differs is μ, which is 0 or 0.9. We use seeds 436, 437, and 438. Before each run we reset the Python, NumPy, and PyTorch random number generators to the seed, so both optimizers start from identical weights. The notebook asserts this on the saved weights. A separate generator with the same seed shuffles the training data, so both optimizers see the same batches in the same order.

The model, the 30 epochs, the learning rate, the batch size, and the 10,000 PCA images follow the authors' notebook `vgg_linear_fit.ipynb`. The original training function uses SGD without momentum, so plain SGD is the original model in our comparison.

C. PCA Basis

We sample 10,000 training images with seed 436 and compute the 27 principal components of their $3 \times 3 \times 3$ patches. The basis uses no test images.

TABLE I
PLAIN SGD (ORIGINAL) COMPARED WITH SGD WITH MOMENTUM 0.9 (OURS). VGG11-BN ON CIFAR-10, 30 EPOCHS, THREE SEEDS, ONE TESLA T4 GPU. VALUES ARE MEAN ± SAMPLE STANDARD DEVIATION. CORRELATIONS ARE BETWEEN FIRST-LAYER ENERGY PROFILES.

Measurement	Plain SGD	Momentum 0.9
Test accuracy (%)	54.54 ± 5.32	78.56 ± 0.98
Training time (s)	587.9 ± 3.6	593.6 ± 4.9
Correlation between seeds	0.92	0.97
lowest to highest	0.89 to 0.97	0.96 to 0.98
Correlation, SGD vs momentum	0.95 to 0.99	

TABLE II
RESULTS OF EACH RUN AFTER EPOCH 30.

Seed	Optimizer	Test acc. (%)	Train acc. (%)	Train loss	Test loss	Time (s)
436	Plain SGD	50.11	93.72	0.182	3.507	590.8
436	Momentum	77.45	99.00	0.031	1.298	598.6
437	Plain SGD	53.06	92.61	0.213	2.640	589.0
437	Momentum	78.95	99.51	0.017	1.212	588.7
438	Plain SGD	60.44	93.52	0.184	2.102	583.8
438	Momentum	79.28	99.45	0.018	1.228	593.4

D. Hardware and Timing

We ran all six runs in one Google Colab session on one NVIDIA Tesla T4 GPU, with Python 3.13.16, PyTorch 2.14.1, and torchvision 0.29.1. The training time of a run is the sum of its 30 per-epoch training times. It excludes the data download, the PCA, and the test evaluation.

E. Metrics

We report the test accuracy after epoch 30 on the 10,000 test images, the training time, and the Pearson correlation between energy profiles. For accuracy and time we give the mean and sample standard deviation over three seeds.

VI. RESULTS

A. Accuracy

Momentum has the higher test accuracy on every seed (Tables I and II). The gain is 27.34, 25.89, and 18.84 percentage points, with a mean of 24.02. The smallest gain is 3.5 times the standard deviation of plain SGD across seeds.

B. Training Time

Plain SGD takes 587.9 s on average and momentum takes 593.6 s, a ratio of 1.01. The difference of 5.7 s is close to the spread across seeds, which is 3.6 s and 4.9 s.

C. Energy Profiles

Between seeds, the profile correlations of plain SGD are 0.970, 0.887, and 0.900. With momentum they are 0.960, 0.965, and 0.982. On the same seed, the correlations between plain SGD and momentum are 0.989, 0.983, and 0.950. In Fig. 1 (left), five of the six profiles peak at component 8. The plain SGD run of seed 438 peaks at component 9, and it is part of the three lowest correlations, 0.887, 0.900, and 0.950.

D. Training Loss

At epochs 5, 10, and 20 the mean training loss of plain SGD is 1.881, 1.401, and 0.578, and the loss with momentum is 0.711, 0.245, and 0.051 (Fig. 1, right). Plain SGD ends at 0.193 and momentum ends at 0.022. Momentum passes the final loss of plain SGD at epoch 12. The final training accuracy is 92.6% to 93.7% for plain SGD and 99.0% to 99.5% with momentum.

VII. DISCUSSION

A. Hypotheses

The data support all four hypotheses. With momentum the profiles stay consistent across seeds, at 0.96 to 0.98 (H1). The two optimizers learn similar profiles on the same seed, at 0.95 to 0.99 (H2). Momentum gains 18.84 to 27.34 points of test accuracy in 30 epochs (H3) and costs 1% more training time (H4).

B. Comparison with the Original Paper

Our seed correlation for plain SGD is 0.92, below the 0.99 in [1]. The plain SGD pair without seed 438 has 0.970, so one run accounts for 0.05 of the 0.07 difference. Our plain SGD runs had not converged. Their mean training loss was 0.193 at epoch 30, against 0.022 with momentum, and the momentum runs reached a seed correlation of 0.97. By (2) the profile depends on the number of steps, so stopping at 30 epochs could lower the correlation. The three plain SGD runs ended at similar training losses, 0.182 to 0.213, so training progress does not explain why seed 438 differs. We did not train plain SGD for longer, so we have not tested this explanation.

The result agrees with the account in [1] that the patch statistics set the profile. Momentum changes the size of each step and leaves the data unchanged. Our experiment does not extend (2) to momentum.

C. Training Speed

Momentum reaches the final training loss of plain SGD in 12 of the 30 epochs, a factor of 2.5. The factor of 10 from (3) holds only when the gradient stays constant between steps, so we expect a smaller factor on minibatch gradients. The extra work per step is one multiplication and one addition per weight, which is consistent with the measured 1% increase in training time.

D. Generalization Gap

Plain SGD has a training accuracy of 92.6% to 93.7% and a test accuracy of 50.11% to 60.44%, with a test loss of 2.10 to 3.51. With momentum the test accuracy is 77.45% to 79.28% and the test loss is 1.21 to 1.30. We did not identify the cause of the larger gap under plain SGD. The accuracy gain in Table I therefore includes this gap as well as the faster decrease of the training loss.

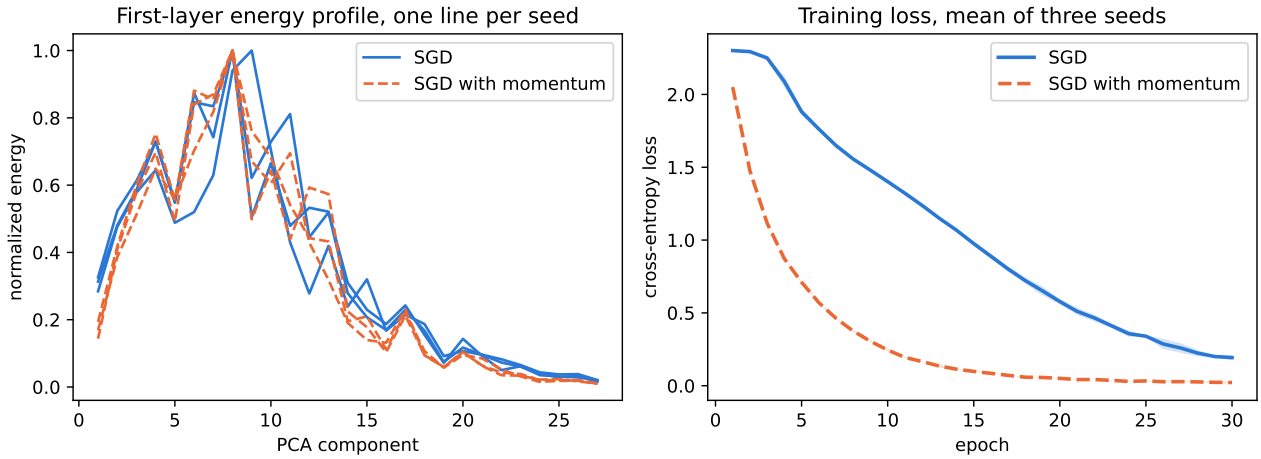


Fig. 1. The left panel shows the first-layer energy profiles of all six runs, one line per seed. The right panel shows the training loss per epoch, averaged over three seeds. The shaded band is one sample standard deviation.

E. Limitations

We used three seeds, one learning rate, one momentum value, one model, one dataset, and 30 epochs. We tuned the learning rate for neither optimizer, so a different learning rate could change the accuracy gap. The training times come from one GPU.

F. Future Work

Training plain SGD until its loss reaches 0.022 would test whether its seed correlation rises from 0.92 toward the 0.99 in [1]. Repeating the six runs on CIFAR-100, where [1] reports 0.97 between seeds, would test a second dataset. Fitting (2) to the profiles of both optimizers and comparing the fitted t would show whether momentum acts like a larger number of plain gradient steps.

G. Conclusion

For VGG11-BN on CIFAR-10, momentum 0.9 raised test accuracy by 24.02 points at 1% more training time. The first-layer energy profile kept a correlation of at least 0.95 with the plain SGD profile on every seed.

Code Availability

Original code:
https://github.com/RheaChowers/CNNs-First-Layer
Our code:
https://github.com/Markus112105/CS436-FinalProj-Fork

References

[1] R. Chowers and Y. Weiss, "What do CNNs learn in the first layer and why? A linear systems perspective," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023.
[2] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013.
[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
[4] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, 2015.
[5] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.
[6] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.
[7] Y. Chen, "PyTorch CIFAR models," GitHub repository. [Online]. Available: https://github.com/chenyaofo/pytorch-cifar-models